

Security Audit Report

Key Threats & Protections for Blockchain-Based Credential Issuance and Verification - v1.0, Prototype Assessment, Aug 2026

1. Overview

EduProof issues and verifies academic credentials using a five-layer architecture: a role-based frontend (Issuer, User/Holder, Verifier), an application/logic layer (AI extraction, hashing, verification, notifications), a Web3 connection layer (ethers.js + MetaMask), a Solidity smart contract on-chain, and off-chain storage (IPFS/cloud/DB) for documents and extracted data.

This report covers the six most relevant attack types for this architecture - what each attack is, how it could target EduProof specifically, and what protects against it in the current design.

1. Unauthorized Credential Issuance [Critical]

What it is: An attacker attempts to create a credential without being a registered, authorized issuer.

How it applies to EduProof: Someone connects a random wallet and calls `issueCredential()` directly on `EduProof.sol`, trying to mint a credential claiming to be from a university that never issued it.

Protection in the app: The smart contract checks `isIssuerAuthorized(address)` before allowing `issueCredential()` to succeed - `registerIssuer()` is the only way an address is added to the authorized-issuer list, so an unregistered wallet is rejected at the contract level.

My additional thinking: I want issuer registration itself to require admin/organization approval, not a self-service signup, so this check actually means something.

2. Issuer Wallet / Private Key Compromise [Critical]

What it is: An attacker gains control of an authorized issuer's wallet (via phishing, malware, or a leaked key) and can now act as that issuer.

How it applies to EduProof: If a university's MetaMask key is stolen, the attacker can call `issueCredential()` or `revokeCredential()` and the blockchain will treat it as a legitimate action from that institution, since authorization is wallet-based.

Protection in the app: EduProof never stores or handles private keys - signing happens entirely inside MetaMask on the issuer's device. `revokeCredential()` also gives a path to invalidate anything issued fraudulently once discovered.

My additional thinking: This is the biggest single point of failure in the design. I'm planning to move toward multisig or an admin-suspend mechanism for issuer wallets so one compromised key can't act alone.

3. Credential Tampering (Hash Mismatch) [High]

What it is: An attacker modifies credential content after issuance - e.g. changing a grade or degree name in the stored document.

How it applies to EduProof: A holder or third party edits the extracted data or the off-chain document, then tries to present it as the original during verification.

Protection in the app: The Hashing Service generates a SHA-256 hash at issuance time and stores it on-chain (`credentialHash`). `verifyCredential()` recomputes the hash of the presented data and compares it to the on-chain value - any change produces a mismatch and the credential is flagged invalid.

My additional thinking: The hash has to be generated from a consistent, canonical version of the data (not the raw AI-extraction output, which could vary slightly) - this needs to be locked down in the Credential Service before the hash is generated.

4. Unauthorized Revocation or Version Update [High]

What it is: Someone who is not the original issuer tries to revoke a credential or push a fake 'corrected' version of it.

How it applies to EduProof: An attacker calls `revokeCredential()` or `createNewVersion()` on a credential hash that belongs to a different issuer, trying to invalidate a legitimate credential or insert a fraudulent updated version.

Protection in the app: `revokeCredential()` and `createNewVersion()` both check that `msg.sender` matches the issuer address originally associated with that credential hash before making any change - a non-owning wallet is rejected.

My additional thinking: I still need to decide whether an institution admin role (separate from the single issuing wallet) should also be allowed to revoke, in case the original issuing employee's wallet is unavailable.

5. Fake / Frontend-Spoofed Verification Result [High]

What it is: An attacker manipulates the frontend or a cloned website so a verifier sees 'VERIFIED' even though the real on-chain record says revoked, invalid, or doesn't exist.

How it applies to EduProof: A fake copy of the verifier page is hosted elsewhere, or client-side JavaScript is altered in the browser, showing a fabricated result without ever actually calling `verifyCredential()` on the real contract.

Protection in the app: Verification always flows through `getCredential()` / `verifyCredential()` on the smart contract - the frontend only displays whatever the contract returns and is never the source of truth. QR codes and credential IDs just point to a lookup, not a pre-decided result.

My additional thinking: I should make it easy for a verifier to double-check independently - e.g. showing the raw credential hash and contract address on the result screen so a suspicious verifier can query the chain directly instead of trusting only my UI.

6. Off-Chain Data Exposure [Medium]

What it is: Personal and credential data stored off-chain (original documents, extracted JSON, student details) is accessed without authorization.

How it applies to EduProof: The off-chain storage layer (IPFS/cloud/DB) is misconfigured, or the metadata URI / CID is exposed and readable by anyone, leaking student names, documents, and extracted personal data.

Protection in the app: By design, only the credential hash, issuer address, timestamp, status, and version go on-chain - personal data and documents stay off-chain specifically so they aren't permanently public. Access to off-chain storage is meant to be role-restricted (Issuer/Holder access, not public).

My additional thinking: This is the one area where the current design is more principle than implemented control - I still need to actually configure private/access-controlled storage (signed URLs or authenticated API access) rather than a public IPFS gateway or open bucket.

2. Summary

Attack	Severity	Core Protection
Unauthorized issuance	Critical	<code>isIssuerAuthorized()</code> check in smart contract
Issuer key compromise	Critical	Keys never leave MetaMask; revocation available if detected
Credential tampering	High	SHA-256 hash comparison on verify
Unauthorized revoke/update	High	<code>msg.sender-is-issuer</code> check on revoke/version functions
Spoofed verification result	High	Verification result always sourced from the contract, not the frontend
Off-chain data exposure	Medium	Personal data kept off-chain; access-restricted storage (in progress)

3. Conclusion

EduProof's security relies on layered checks rather than the blockchain alone: authorized-issuer verification, ownership checks on revoke/update, cryptographic hashing for tamper detection, and keeping the frontend as a display layer only - never the source of truth. The main open risk is issuer wallet security and locking down access to off-chain storage, both noted above as active next steps.